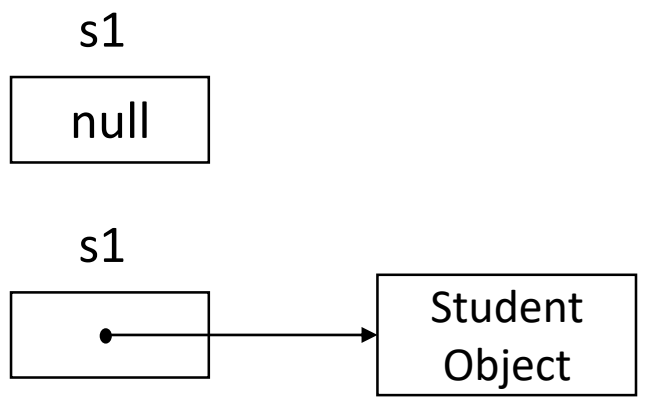
Class and Object

Class & Object

- Everything in Java is associated with classes and objects, along with its attributes and methods.
- For example: in real life, a car is an object. The car has **attributes**, such as weight and color, and **methods**, such as drive and brake.
- It is a user-defined blueprint or prototype from which objects are created. E.g. Student is a class while a particular student named Ravi is an object.
- A Class in Java may contain:
  - Data member, Method, Constructor, Nested Class, Interface
- Objects are the instances of a class that are created to use the attributes and methods of a class.
- When an object of a class is created, the class is said to be **instantiated.**

Example

```
class Student {
int id;
String name;
    public static void main(String args[])
    {
        Student s1;                    //Declaration
        s1 = new Student();            //Instantiate
        System.out.println(s1.id);
        System.out.println(s1.name);
    }
}
```



- Create multiple objects:

  Student S1=new Student();

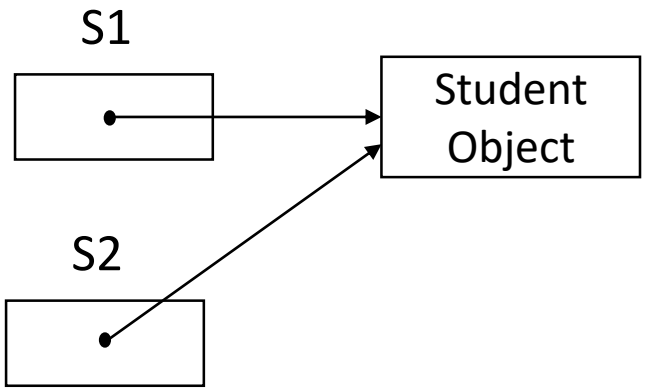
  Student S2=new Student();

  …. So on

- It is possible to create multiple objects share the same reference.

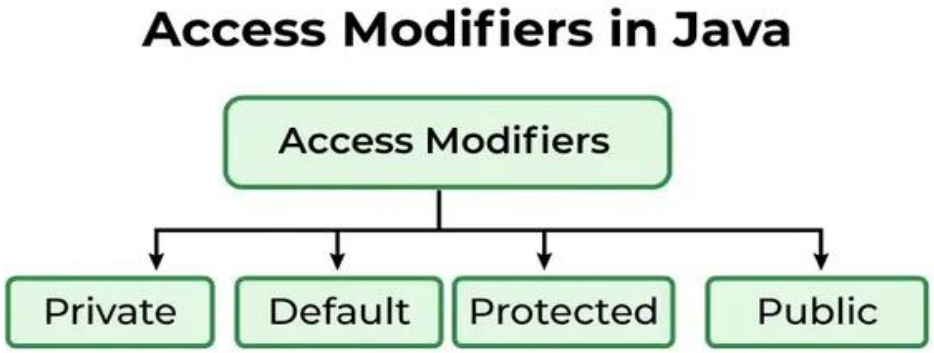
  Student S1=new Student();

  Student S2=S1;



Access Modifier

- Access modifiers help to restrict the scope of a class, constructor, variable, method, or data member.



Modifier	**Description**
public	The code is accessible for all classes
private	The code is only accessible within the declared class
default	The code is only accessible in the same package. This is used when you don't specify a modifier.
protected	The code is accessible in the same package and **subclasses**.

Access Modifier

	Default	Private	Protected	Public
Same Class	Yes	Yes	Yes	Yes
Same Package Subclass	Yes	No	Yes	Yes
Same Package Non-Subclass	Yes	No	Yes	Yes
Different Package Subclass	No	No	Yes	Yes
Different Package Non-Subclass	No	No	No	Yes

```
class Book {
String title;
   String author;

public void setTitle(String bookTitle) {
      title = bookTitle;
   }

public void setAuthor(String bookAuthor) {
      author = bookAuthor;
   }

public void displayDetails() {
      System.out.println("Title: " + title);
      System.out.println("Author: " + author);
   }
}
```

```
public class BookDemo {
   public static void main(String[] args) {

     Book book1 = new Book();
      Book book2 = new Book();
     book1.setTitle(“Java: The Complete Reference");
      book1.setAuthor("Herbert Schildt");
      book2.setTitle("Head First Java");
      book2.setAuthor("Kathy Sierra");
     System.out.println("Details of book1:");
      book1.displayDetails();

      System.out.println("\nDetails of book2:");
      book2.displayDetails();
   }
}
```

Method overloading

- **Method overloading** allows a class to have more than one method with the same name, but different parameter lists (i.e., different number or types of parameters).
- It is ***Compile-time Polymorphism, Static Polymorphism, or Early binding***.
- **Different Ways of Method Overloading in Java**
  - Changing the Number of Parameters.
  - Changing Data Types of the Arguments.
  - Changing the Order of the Parameters of Methods
- Key Rules:
  - Same method name
  - Methods must differ in the number or types of parameters (method signature).
  - The **return type** can be different but is not considered part of the method signature for overloading.
  - Method overloading can’t be done by changing the return type alone.

Example

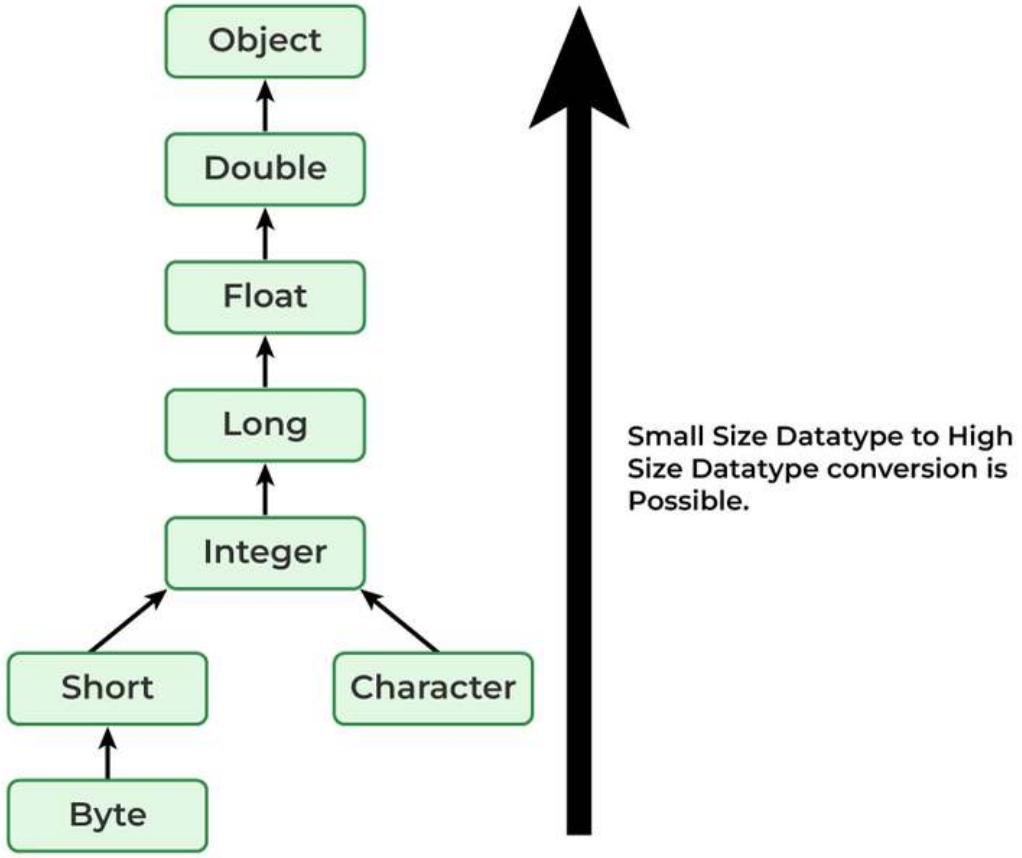
```
class Calculator {
public int add(int a, int b) {
        return a + b;
    }
public int add(int a, int b, int c) {
        return a + b + c;
    }
public double add(double a, double b) {
        return a + b;
    }
public double add(int a, double b) {
        return a + b;
    }
}
```

```
public class MethodOverloadingDemo {
    public static void main(String[] args) {
        Calculator calc = new Calculator();


        // Calling the different overloaded add methods
        System.out.println("Sum of two integers: " +
calc.add(10, 20));              // Calls add(int, int)
        System.out.println("Sum of three integers: " +
calc.add(10, 20, 30));    // Calls add(int, int, int)
        System.out.println("Sum of two doubles: " +
calc.add(5.5, 7.3));        // Calls add(double, double)
        System.out.println("Sum of an int and a double: " +
calc.add(10, 7.8));   // Calls add(int, double)
    }
}
```

What if the exact prototype does not match with arguments?

- Priority-wise, the compiler takes these steps:
  - Type Conversion but to a higher type(in terms of range) in the same family.
  - Type conversion to the next higher family(suppose if there is no long data type available for an int data type, then it will search for the float data type).



Example

```
class Demo {
        public void show(int x) {
                System.out.println("In int" + x);
        }
        public void show(String s) {
                System.out.println("In String" + s);
        }
        public void show(byte b) {
                System.out.println("In byte" + b);
        }
}
```

```
class UseDemo {
        public static void main(String[] args)
{
                byte a = 25;
                Demo obj = new Demo();
                obj.show(a);
                obj.show("hello");
                obj.show(250);
                obj.show('A');

                obj.show("A");

                obj.show(7.5);
        }
}
```

Static method

- Any method that uses the static keyword is referred to as a static method.
- A static method in Java is a method that is part of a class rather than an instance of that class.
- Every instance of a class has access to the method.
- Static methods have access to class variables (static variables) without using the class's object (instance).
- Only static data may be accessed by a static method. It is unable to access data that is not static (instance variables).

```
Access_modifier static void methodName()
{
} // Method body.
```

Syntax to call a static method: className.methodName();

The static method does not have access to the instance variable

```
import java.io.*;
public class Test {
static int a = 40;
int b = 50;
   void simpleDisplay()
   {
      System.out.println(a);
      System.out.println(b);
   }
static void staticDisplay()
   {
System.out.println(a);
   }
public static void main(String[] args)
   {

Test obj = new Test();
      obj.simpleDisplay();staticDisplay();
   }
}
```

In both static and non-static methods, static methods are directly accessed.

```
import java.io.*;
public class StaticExample {
   static int num = 100;
    static String str = “NIT Patna";
static void display()
   {
      System.out.println("static number is " + num);
      System.out.println("static string is " + str);
   }
void nonstatic()
   {
display();
   }
public static void main(String args[])
   {
      StaticExample obj = new StaticExample();
     obj.nonstatic();
display();
   }
}
```

- **Why use Static Methods?**
  - To access and change static variables and other non-object-based static methods.
  - Utility and assist classes frequently employ static methods.
- **Restrictions in Static Methods:**
  - Non-static data members or non-static methods cannot be used by static methods, and static methods cannot call non-static methods directly.
  - In a static environment, this and super aren’t allowed to be used.
- **Why is the main method in Java static?**
  - It’s because calling a static method isn’t needed of the object. If it were a non-static method, JVM would first build an object before calling the main() method, resulting in an extra memory allocation difficulty.

Difference Between the static method and instance method

Instance Methods	Static Methods
It requires an object of the class.	It doesn’t require an object of the class.
It can access all attributes of a class.	It can access only the static attribute of a class.
The methods can be accessed only using object reference.	The method is only accessed by class name.
Syntax: Objref.methodname()	Syntax: className.methodname()

‘this’ keyword

- In Java, the this keyword is a special reference variable used within an instance method or constructor to refer to the current object.
- It helps to distinguish between instance variables and parameters or local variables with the same name
- Key Uses of this Keyword
  - *Refer to Instance Variables:* When local variables or parameters have the same name as instance variables, this is used to refer to the instance variables.
  - *Invoke Other Constructors:* this can be used to call another constructor in the same class (constructor chaining).
  - *Pass the Current Object:* this can be passed as an argument to methods or constructors of other classes.
  - *Return the Current Object:* this can be used to return the current object from a method, often used in method chaining.

Example

```
class Person {
    String name;
    int age;
    public void setDetails(String name, int age) {
        this.name = name;  // Refers to the instance variable 'name'
        this.age = age;    // Refers to the instance variable 'age'
    }
public void displayInfo() {
        System.out.println("Name: " + this.name + ", Age: " + this.age);
    }
}
public class Main {
    public static void main(String[] args) {
    Person person = new Person();
   person.setDetails("Alice", 30);
    person.displayInfo();  // Output: Name: Alice, Age: 30
    }
}
```

Questions

```
class Test {
    static int x = 10;
    public static void main(String[] args) {
        Test t1 = new Test();
        Test t2 = new Test();
        t1.x = 20;
        System.out.println(t2.x);
    }
}
```

```
class Test {
    static int x = 10;
    static void modify() {
        x += 5;
    }
    public static void main(String[] args) {
        Test obj1 = new Test();
        Test obj2 = new Test();
        obj1.modify();
        System.out.println(obj2.x);
    }
}
```

```
class Test {
    public static void main(String[] args) {
        int i = 0;
        while (i < 5) {
            if (i % 2 == 0)
                continue;
            System.out.print(i + " ");
            i++;
        }
    }
}
```

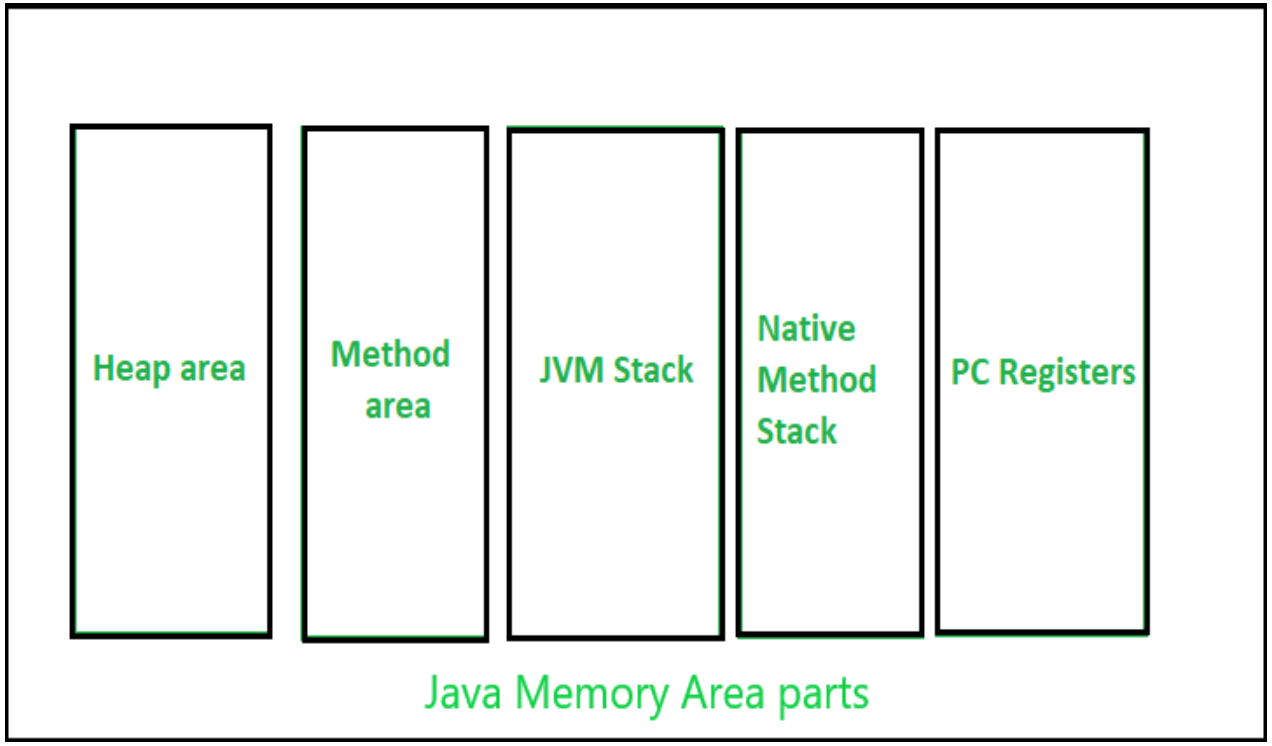
```
class Test {
    public static void main(String[] args)
{
        for (int i = 0; i < 3;
System.out.print(i + " "), i++);
    }
}
```

```
class Test {
    public static void main(String[] args) {
        int a = 5;
        System.out.println(a++ + ++a);
    }
}
```

```
class Test {
    public static void main(String[] args) {
        int a = 4;
        int b = a++ * --a + ++a - a--;
        System.out.println(b);
    }
}
```

Java Memory Structure

- The major concepts in Java Memory Management :
  - JVM Memory Structure
  - Garbage Collector
- **Java Memory Structure:**
- Some of the areas are created by the JVM whereas some are created by the threads that are used in a program.
- memory area created by JVM is destroyed only when the JVM exits.
- The data areas of thread are created during instantiation and destroyed when the thread exits.



- **Heap :**
  - stores the actual object in a memory.
  - Heap can be of fixed or dynamic size depending upon the system's configuration.
  - The user can control the heap.
  - When a new keyword is used, object is assigned a space in heap, but the reference of the same exists onto the stack.
  - *Scanner sc = new Scanner(System.in);*
  - The above statement creates the object of Scanner class which gets allocated to heap whereas the reference 'sc' gets pushed to the stack.
- ***Note:*** *Garbage collection in heap area is mandatory.*

- **Method Area (Metaspace):**
  - It is a logical part of the heap area and is created on virtual machine startup.
  - This memory is allocated for the following:
    - Bytecode: The compiled bytecode from .class files.
    - Class Metadata: Information about class methods, fields, and constants.
    - Static Variables: Static variables are also stored here.
  - Can be of a fixed size or expanded as required by the computation.
  - Needs not to be contiguous.

- **JVM Stacks:**
  - A stack is created at the same time when a thread is created and is used to store data and partial results which will be needed while returning value for method and performing dynamic linking.
  - Stores method **call frames (activation records)**: local variables, parameters, return addresses.
  - Stacks can either be of fixed or dynamic size.
  - The memory for stack needs not to be contiguous.
- **Native method Stacks:**
  - It is used for native code written in non-Java languages such as C or C++. When Java invokes native methods, their execution is managed in this stack, rather than the Java stack.

- **Program counter (PC) registers:**
- Each thread has its own **PC register**, which keeps track of the current execution point within the thread. **Program Counter** is a **register** that stores the **address of the next instruction** to be executed by the **Java Virtual Machine (JVM)**.

```
class Example {
    static int x = 10;       // Stored in Method Area (Metaspace)
    int y;      // Instance variable, stored in Heap
    public Example(int y) {
        this.y = y;
    }
    public static void staticMethod() {
        System.out.println("Static method called");
    }
    public void instanceMethod() {
        int localVariable = 20;           // Local variable, stored in Stack
        System.out.println("Instance method called: " + y + ", " + localVariable);
    }
    public static void main(String[] args) {
        Example obj = new Example(30); /         / obj stored in Heap, reference stored in Stack
        obj.instanceMethod();          // Method execution creates a stack frame with local variables
        Example.staticMethod();       // Static method executed without instance
    }
}
```

Memory Allocation Process:

Method Area (Metaspace): The static variable x and the class-level information of Example are stored in the Metaspace.

Heap: The object obj is created on the heap, and its instance variable y is stored here as well.

Stack: The reference to the object obj and the local variable localVariable in the instanceMethod are stored in the stack.

Garbage collection

- In Java, memory management is primarily handled through **automatic garbage collection**, which means that the developer doesn't need to explicitly manage memory allocation and deallocation.
- The **garbage collector (GC)** is a part of the Java **JVM (Java Virtual Machine)** responsible for automatically reclaiming memory that is no longer in use by the program, i.e., memory occupied by objects that are no longer reachable by any active part of the application.
- Garbage collection techniques include:
- **Minor GC**: Focuses on the **Young Generation.**
  - The young generation is where new objects are allocated.
- **Major GC**: Focuses on the **Old Generation**.
  - If an object survives several garbage collection cycles, it is promoted to the **Old Generation**.
  - This region holds objects that have survived several garbage collection cycles in the young generation and are considered long-lived.

Note: System.gc() *and* ***Runtime.gc()*** *are the methods which requests for Garbage collection to JVM explicitly but it doesn't ensures garbage collection as the final decision of garbage collection is of JVM only.*

Constructor

- A constructor in Java is a **special method** that is used to initialize objects.
- Every time an object is created using the new() keyword, at least one constructor is called.
- The constructor(s) of a class must have the same name as the class name in which it resides.
- Access modifiers can be used in constructor declaration to control its access i.e which other class can call the constructor.

```
class Person {
   String name;
   int age;
Public void setData(String name, int age) {
       this.name = name;
       this.age = age;
}
public void displayInfo() {
       System.out.println("Name: " + name + ", Age: " + age);
   }
}
public class Main {
   public static void main(String[] args) {
       Person person1 = new Person(); person1.setData("Alice", 25);
       Person person2 = new Person(); person2.setData ("Bob", 30);
person1.displayInfo();        // Output: Name: Alice, Age: 25
       person2.displayInfo();                // Output: Name: Bob, Age: 30
   }
}
```

Types of Constructors in Java

- Default Constructor
- Parameterized Constructor
- Copy Constructor
- **Default Constructor**
  - A constructor that has no parameters is known as default the constructor.
  - The default constructor can be implicit or explicit.
  - If we don’t define explicitly, we get an implicit default constructor. If we manually write a constructor, the implicit one is overridden.
  - ***Note:*** *Default constructor provides the default values to the object like 0, null, etc. depending on the type.*

Default constructor

```
class Car {
    String model;
    int year;
public Car() {
        model = "Unknown";
        year = 2000;
    }
    public void displayInfo() {
        System.out.println("Car Model: " + model + ", Year: " + year);
    }
}
public class Main {
    public static void main(String[] args) {
         Car myCar = new Car();
        myCar.displayInfo();                // Output: Car Model: Unknown, Year: 2000
    }
}
```

Parametrized constructor

- A constructor that has parameters is known as parameterized constructor.
- If we want to initialize fields of the class with our own values, then use a parameterized constructor.
- **Constructors do not return values**.

```
class Car {
    String model;
    int year;
public Car(String model, int year) {
        this.model = model;
        this.year = year;
    }
public void displayInfo() {
        System.out.println("Car Model: " + model + ", Year: " + year);
    }
}
public class Main {
    public static void main(String[] args) {
        Car myCar = new Car("Toyota", 2021);
        myCar.displayInfo();  // Output: Car Model: Toyota, Year: 2021
    }
}
```

Constructor overloading

- Just like methods, we can overload constructors for creating objects in different ways.
- The compiler differentiates constructors on the basis of the number of parameters, types of parameters, and order of the parameters.
- Constructor overloading allows you to create objects in different ways (with different sets of initial values), making it more flexible and easier to handle different situations during object instantiation.
- When there is no exact match for the constructor parameters provided during object creation, the Java compiler will try to promote smaller data types to a larger compatible type, similar to method overloading.

```
import java.io.*;
class Geek {
Geek(String name) {
        System.out.println("Constructor with one " + "argument - String : " + name);          }
Geek(String name, int age){
        System.out.println( "Constructor with two arguments : "  + " String and Integer : " + name + " " + age);       }
Geek(long id) {
        System.out.println("Constructor with one argument : "+ "Long : " + id);         }
}
class Test {
    public static void main(String[] args)  {
Geek geek2 = new Geek("Shikhar");
Geek geek3 = new Geek("Dharmesh", 26);
Geek geek4 = new Geek(325614567);
    }
}
```

Call of one constructor to another constructor

- In Java, one constructor can be called from another constructor in the same class using the ‘this’ keyword. This is known as constructor chaining.
- It allows the reuse of constructor code.

```
class Temp{
        Temp(){
                this(5);            // calls constructor 2
        System.out.println("The Default constructor");
        }
        Temp(int x){         //constructor 2
                this(5, 15);   // calls constructor 3
                System.out.println(x);
        }
        Temp(int x, int y){         //constructor 3
                System.out.println(x * y);
        }
        public static void main(String args[])
        {
        // invokes default constructor first
        new Temp();
        }
}
```

Copy Constructor

- A **copy constructor** in Java is a special type of constructor used to create a new object as a copy of an existing object.
- This concept is borrowed from C++ and is not built into Java like other languages, but it can be implemented manually.
- *we can create our own copy constructor by passing the object of the same class to the other instance(object) of the class.*
- **Purpose of a Copy Constructor**
  - **Create a Duplicate**: It allows you to create a new object that is a copy of an existing object, including its state (i.e., the values of its attributes).
  - **Deep Copy**: In cases where an object contains references to other objects, a copy constructor can be used to perform a deep copy, ensuring that the new object and the original object do not share references to mutable objects.

Copy Constructor

```
class Person {
    String name;
    int age;
public Person(String name, int age) {
        this.name = name;
        this.age = age;
    }
public Person(Person other) {
        this.name = other.name;  // Copy the name
        this.age = other.age;    // Copy the age
    }
public void displayInfo() {
System.out.println("Name: " + name + ", Age: " +
age);
    }
}
```

```
public class Main {
    public static void main(String[] args) {
   Person person1 = new Person("Alice", 30);
   Person person2 = new Person(person1);
    System.out.println("Original Person:");
        person1.displayInfo();
        System.out.println();
        System.out.println("Copied Person:");
        person2.displayInfo();
    }
}
```

Deep copy (using copy constructor)

```
class Address {
    String city;

    Address(String city) {
        this.city = city;
    }

    // Copy Constructor (Deep Copy)
    Address(Address other) {
        this.city = other.city;
    }
}
```

```
class Person {
    String name;
    Address address;
    Person(String name, Address address) {
        this.name = name;
        this.address = address;
    }
    // Copy Constructor (Deep Copy)
    Person(Person other) {
        this.name = other.name;
        this.address = new Address(other.address); // deep copy of Address
    }
}
public class Main {
    public static void main(String[] args) {
        Person p1 = new Person("Alice", new Address("Delhi"));
        Person p2 = new Person(p1);  // Deep copy

        p2.address.city = "Mumbai";

        System.out.println(p1.address.city); // Output: Delhi
        System.out.println(p2.address.city); // Output: Mumbai
    }
}
```

Wrapper class

- In Java, wrapper classes are used to convert primitive data types (like int, char, etc.) into objects.
- Why we need Wrapper class?
  - They convert primitive data types into objects. Objects are needed if we wish to modify the arguments passed into a method (because primitive types are passed by value).
  - This allows primitive values to be treated as objects, which is necessary when working with collections (like ArrayList and HashMap)
  - The classes in java.util package handles only objects and hence wrapper classes help in this case also.
  - An object is needed to support synchronization in multithreading.

Primitive Data Types and their Corresponding Wrapper Class

Primitive Data Type	Wrapper Class
char	Character
byte	Byte
short	Short
int	Integer
long	Long
float	Float
double	Double
boolean	Boolean

Features of Wrapper class

1. Java automatically converts between primitive types and their corresponding wrapper objects.
   - **Autoboxing**
     - The automatic conversion of primitive types to the object of their corresponding wrapper classes is known as autoboxing. For example – conversion of int to Integer, long to Long, double to Double, etc.

```
int x = 10;
Integer obj = x; // Autoboxing: int to Integer
```

   - **Unboxing**
     - Converting a wrapper object back into its corresponding primitive type.

```
Integer obj = 20;
int y = obj; // Unboxing: Integer to int
```

Features of Wrapper class

1. Useful Methods:Wrapper classes provide various methods to manipulate and convert data. For example:
   - Integer.parseInt("123"): Converts a string to an int.
   - Double.toString(10.5): Converts a double to a String.
2. Immutable Objects:
   Wrapper objects are immutable, meaning their values cannot be changed after they are created.
3. Usage in Collections: Since collections like ArrayList, HashMap, etc., only work with objects, wrapper classes allow the use of primitive values in these structures.
   - ArrayList<Integer> list = new ArrayList<>();
   - list.add(10); // Autoboxing: int to Integer

Example

```
public class WrapperExample {
    public static void main(String[] args) {
        int num = 100;
        Integer obj = num;          // Autoboxing: primitive to object
         int value = obj;           // Unboxing: object to primitive
        String str = Integer.toString(num);
        int parsedValue = Integer.parseInt("123");

        System.out.println("Object value: " + obj);
        System.out.println("Primitive value: " + value);
        System.out.println("Parsed value: " + parsedValue);
    }
}
```

Example

```
import java.util.ArrayList;
import java.util.HashMap;
public class WrapperClassInCollections {
    public static void main(String[] args) {
ArrayList<Integer> integerList = new ArrayList<>();
integerList.add(10); // autoboxed to Integer
        integerList.add(20);
        integerList.add(30);
System.out.println("ArrayList contents:");
        for (Integer num : integerList) {
            System.out.println(num);
        }
HashMap<String, Double> priceMap = new HashMap<>();
        // Adding key-value pairs (autoboxing will occur for Double)
        priceMap.put("Apple", 1.99);
        priceMap.put("Banana", 0.59);
        priceMap.put("Cherry", 2.99);

        // Accessing and printing values
        System.out.println("\nHashMap contents:");
        for (String key : priceMap.keySet()) {
            Double price = priceMap.get(key); // unboxing occurs here
            System.out.println(key + ": $" + price);
        }
    }
}
```

```
import java.io.*;
class GFG {
public static void main(String[] args){
byte a = 1;
Byte byteobj = new Byte(a);
int b = 10;
Integer intobj = new Integer(b);
float c = 18.6f;
Float floatobj = new Float(c);
double d = 250.5;
Double doubleobj = new Double(d);
char e = 'a';
Character charobj = e;
System.out.println( "Values of Wrapper objects (printing as objects)");
System.out.println("\nByte object byteobj: "+ byteobj);
        System.out.println("\nInteger object intobj: "+ intobj);
        System.out.println("\nFloat object floatobj: " + floatobj);
        System.out.println("\nDouble object doubleobj: "+ doubleobj);
        System.out.println("\nCharacter object charobj: "+ charobj);
        // objects to data types (retrieving data types from objects)
unwrapping objects to primitive data types
        byte bv = byteobj;
        int iv = intobj;
        float fv = floatobj;
        double dv = doubleobj;
        char cv = charobj;
System.out.println("\nUnwrapped values (printing as data
types)");
        System.out.println("\nbyte value, bv: " + bv);
        System.out.println("\nint value, iv: " + iv);
        System.out.println("\nfloat value, fv: " + fv);
        System.out.println("\ndouble value, dv: " + dv);
        System.out.println("\nchar value, cv: " + cv);
    }
}
```

Array

- **Arrays** are fundamental structures in Java that allow us to store multiple values of the same type in a single variable.
- Arrays in Java work differently than they do in C/C++. Arrays are objects in Java.
- In Java, all arrays are **dynamically allocated**.
- An array can contain primitives (int, char, etc.) and object (or non-primitive) references of a class, depending on the definition of the array. In the case of primitive data types, the actual values might be stored in contiguous memory locations (JVM does not guarantee this behavior). In the case of class objects, the actual objects are stored in a heap segment.
  - **Array Declaration**
    - // Declaration without size (size will be specified later)
    - int[] myArray; // or int myArray[];
    - // Declaration and creation of an array with size 5
    - int[] myArray = new int[5];
  - **Initializing an Array**
    - // Initializing while declaring
    - int[] myArray = {10, 20, 30, 40, 50};
    - // Creating an array and then assigning values
    - int[] myArray = new int[5]; // Array of size 5 with default values (0)
    - myArray[0] = 10; // Assigning values to elements
    - myArray[1] = 20;
  - **Array Length**
    - int length = myArray.length;

Example of Array Declaration, Initialization, and Usage:

```
public class ArrayExample {
    public static void main(String[] args) {
    int[] numbers = {1, 2, 3, 4, 5};                                    // Declare and initialize an array of integers
     System.out.println("First element: " + numbers[0]);              // Output: 1
       System.out.println("Third element: " + numbers[2]);          // Output: 3
       // Modifying an element
       numbers[1] = 20;         // Change the second element
       System.out.println("Modified array: ");
       // Looping through the array and printing all elements
       for (int i = 0; i < numbers.length; i++) {
          System.out.println("Element at index " + i + ": " + numbers[i]);
       }
    }
}
```

Class of Array

```
class Test {
   public static void main(String[] args) {
      int[] arr = new int[5];
      System.out.println(arr.getClass().getName());
// Prints array class type
      System.out.println(arr instanceof Object);    //
Checks if array is an Object
   }
}
```

O/P: [I
true

- arr.getClass().getName() prints [I, meaning 1D array of integers (I for int).
- arr instanceof Object returns true, proving that arrays are instances of Object.

- Even though arrays in Java are instances of Object, when we call getClass().getName(), we see output like [I instead of "java.lang.Object". This is due to how Java represents array types internally.
- Java uses a special syntax for **array type names**. The notation follows the **JVM internal representation**:

Array Type	**JVM Representation (getClass().getName())**
int[]	[I
double[]	[D
boolean[]	[Z
char[]	[C
String[]	[Ljava.lang.String;
int[][]	[[I

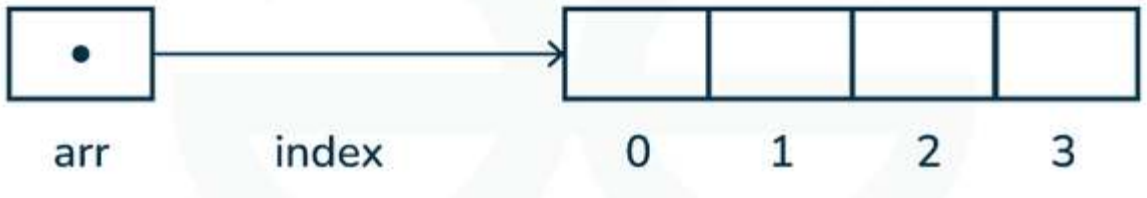
Types of Array

- Single Dimensional Array
- Multidimensional Array

- Note: JVM throws **ArrayIndexOutOfBoundsException** to indicate that the array has been accessed with an illegal index.

Single Dimension Array

- These are the most common type of arrays, where elements are stored in a linear order.
- int[] singleDimArray = {1, 2, 3, 4, 5}; // A single-dimensional array



Multi-Dimensional Arrays

- In Java, multi-dimensional arrays are arrays of arrays.
- They allow you to store data in more than one dimension, such as in a grid or matrix.
- The most common type of multi-dimensional array is the two-dimensional array, but Java supports arrays with more dimensions.
- int[][] matrix = {

  {1, 2, 3, 4},

  {5, 6, 7, 8},

  {9, 10, 11, 12}

}; // Creates and initializes a 2D array with specified values

2D Array example

```
public class Simple2DArrayExample {
    public static void main(String[] args) {
        // Create and initialize a 2x3 matrix
        int[][] matrix = {
            {1, 2, 3},
            {4, 5, 6}
        };

        // Print the matrix
        System.out.println("Matrix:");
        for (int i = 0; i < matrix.length; i++) {
            for (int j = 0; j < matrix[i].length; j++) {
                System.out.print(matrix[i][j] + " ");
            }
            System.out.println(); // Move to the next line after each row
        }
    }
}
```

Multi-Dimensional Arrays with More Than Two Dimensions

3 D Array:

```
int[][][] threeDArray = new int[3][4][5]; // Creates a 3D array with 3 layers, 4 rows, and 5 columns
int[][][] threeDArray = {
    {
        {1, 2, 3, 4, 5},
        {6, 7, 8, 9, 10},
        {11, 12, 13, 14, 15},
        {16, 17, 18, 19, 20}
    },
    {
        {21, 22, 23, 24, 25},
        {26, 27, 28, 29, 30},
        {31, 32, 33, 34, 35},
        {36, 37, 38, 39, 40}
    },
    {
        {41, 42, 43, 44, 45},
        {46, 47, 48, 49, 50},
        {51, 52, 53, 54, 55},
        {56, 57, 58, 59, 60}
    }
};
```

Arrays of Objects

```
class Student {
    public int roll_no;
    public String name;
    Student(int roll_no, String name)
    {
        this.roll_no = roll_no;
        this.name = name;
    }
}
// Elements of the array are objects of a class Student.
public class GFG {
    public static void main(String[] args)
    {
        // declares an Array of Student
        Student[] arr;

        // allocating memory for 5 objects of type Student.
        arr = new Student[5];

        // initialize the first elements of the array
        arr[0] = new Student(1, "aman");

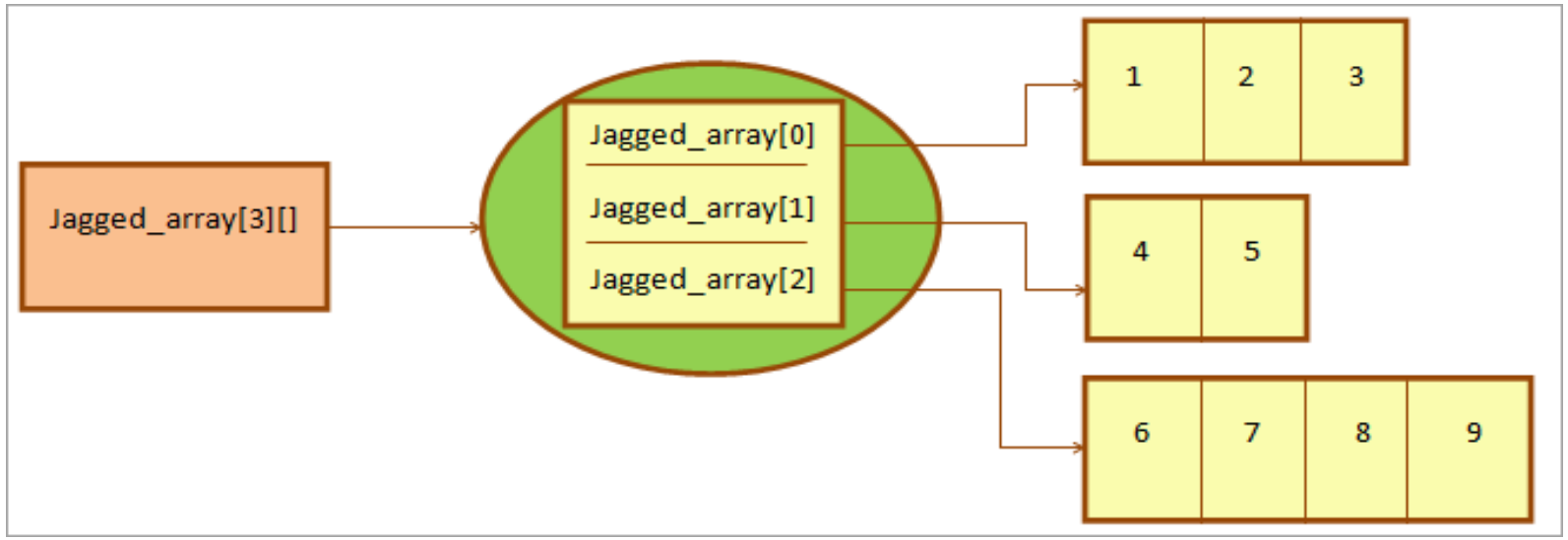
        // initialize the second elements of the array
        arr[1] = new Student(2, "vaibhav");

        // so on...
        arr[2] = new Student(3, "shikar");
        arr[3] = new Student(4, "dharmesh");
        arr[4] = new Student(5, "mohit");

        // accessing the elements of the specified array
        for (int i = 0; i < arr.length; i++)
            System.out.println("Element at " + i + " : "
                               + arr[i].roll_no + " "
                               + arr[i].name);
    }
}
```

Jagged Array

- A jagged array is an array of arrays such that member arrays can be of different sizes. i.e., we can create a 2-D array but with a variable number of columns in each row.



Sample Program

```
class TestJaggedArray{
    public static void main(String[] args){
        //declaring a 2D array with odd columns
        int arr[][] = new int[3][];
        arr[0] = new int[3];
        arr[1] = new int[4];
        arr[2] = new int[2];
        //initializing a jagged array
        int count = 0;
        for (int i=0; i<arr.length; i++)
            for(int j=0; j<arr[i].length; j++)
                arr[i][j] = count++;
        //printing the data of a jagged array
        for (int i=0; i<arr.length; i++){
            for (int j=0; j<arr[i].length; j++){
                System.out.print(arr[i][j]+" ");
            }
            System.out.println();//new line
        }
    }
}
```

Passing Arrays to Methods

- Like variables, we can also pass arrays to methods.

```
public class Test {
public static void sum(int[] arr) {
int sum = 0;
      for (int i = 0; i < arr.length; i++)
         sum += arr[i];

      System.out.println("sum of array values : " + sum);
   }
public static void main(String args[]) {
      int arr[] = { 3, 1, 2, 5, 4 };

      // passing array to method method sum
      sum(arr);
   }
}
```

Returning Arrays from Methods

- As usual, a method can also return an array.

```
class Test {

    public static int[] m1()
    {
          int [] ar={1,2,3};
          return ar;
    }
public static void main(String args[])
    {
       int arr[] = m1();

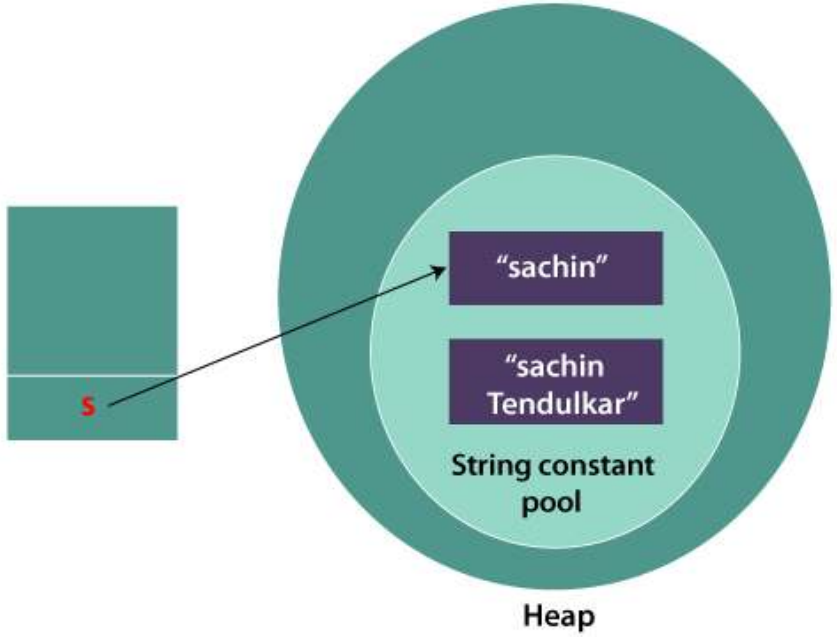
       for (int i = 0; i < arr.length; i++)
          System.out.print(arr[i] + " ");
    }
}
```

String

- In Java, string is basically an object that represents sequence of char values.
  - String name = "Geeks"; //Also called string literal
- The java.lang.String class is used to create a string object.
- A string acts the same as an array of characters in Java.
  - char[] ch={'j','a','v','a'};
  - String s=**new** String(ch); //convert a char array to string
  - Above is same as String s="java"
- We can also create string like this: String s=new String("Java");
- In Java, objects of String are immutable which means a constant and cannot be changed once created.

String are immutable

```
class Testimmutablestring{
 public static void main(String args[]){
   String s="Sachin";
   s.concat(" Tendulkar");//concat() method appends the string at the end
   System.out.println(s);//will print Sachin because strings are immutable objects
 }
}
```



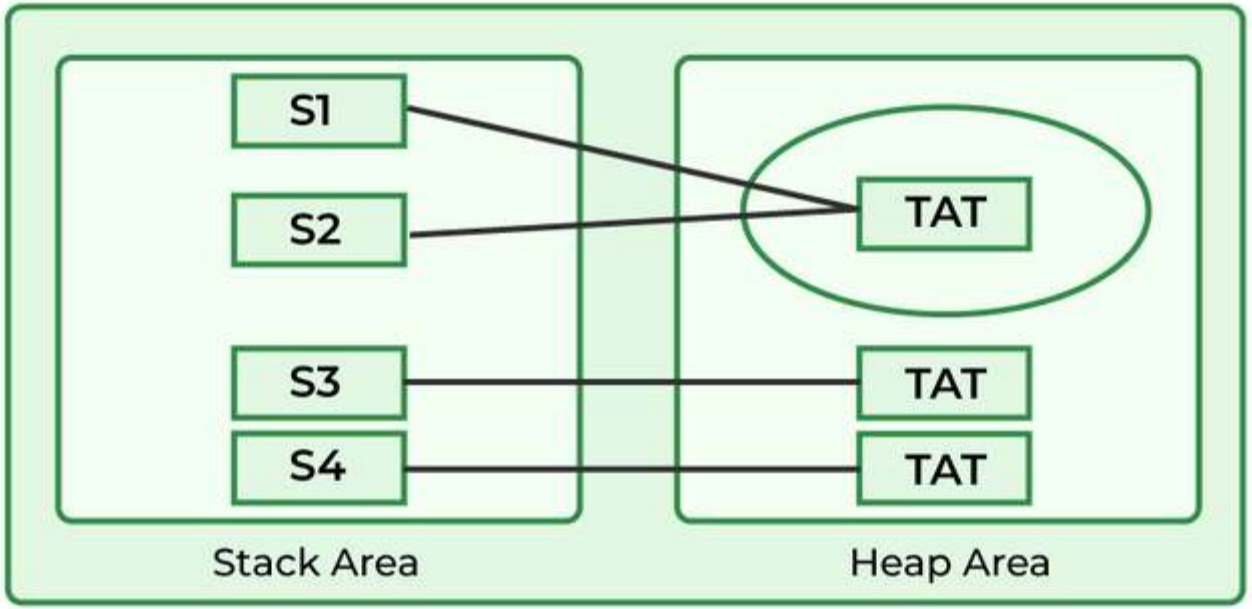
Memory Allotment of String

- Whenever a String Object is created as a literal, the object will be created in the **String constant pool**. This allows JVM to optimize the initialization of String literal.
  - String demoString = “Java";
- The string can also be declared using a **new** operator i.e. dynamically allocated. In case of String are dynamically allocated they are assigned a new memory location in the heap. This string will not be added to the String constant pool.
  - String demoString = new String("Geeks");
  - String internedString = demoString.intern();
  - // this will add the string to string constant pool.

Memory Allotment of String

```
class StringStorage {
    public static void main(String args[])
    {
        // Declaring Strings
        String s1 = "TAT";
        String s2 = "TAT";
        String s3 = new String("TAT");
        String s4 = new String("TAT");

        // Printing all the Strings
        System.out.println(s1);
        System.out.println(s2);
        System.out.println(s3);
        System.out.println(s4);
    }
}
```



Methods of String class

- int length()
  - Returns the number of characters in the String.
  - "GeeksforGeeks".length(); // returns 13
- Char charAt(int i)
  - Returns the character at i^{th} index.
  - "GeeksforGeeks".charAt(3); // returns ‘k’
- String substring (int i)
  - Return the substring from the ith index character to end.
  - "GeeksforGeeks".substring(3); // returns “ksforGeeks”
- String substring (int i, int j)
  - Returns the substring from i to j-1 index.
  - "GeeksforGeeks".substring(2, 5); // returns “eks”
- String concat(String str)
  - Concatenates specified string to the end of this string.
  - String s1 = ”Geeks”;
  - String s2 = ”forGeeks”;
  - String output = s1.concat(s2); // returns “GeeksforGeeks”

Methods of String class

- int indexOf(String s)
  - Returns the index within the string of the first occurrence of the specified string.
  - If String s is not present in input string then -1 is returned as the default value.
  - String s = ”Learn Share Learn”;
  - int output = s.indexOf(“Share”); // returns 6
  - String s = "Learn Share Learn“
  - int output = s.indexOf(“Play”); // return -1
- int indexOf(String s, int i)
  - Returns the index within the string of the first occurrence of the specified string, starting at the specified index.
  - String s = ”Learn Share Learn”;
  - int output = s.indexOf("ea",3);// returns 13
- Int lastIndexOf(String s)
  - Returns the index within the string of the last occurrence of the specified string.
  - If String s is not present in input string then -1 is returned as the default value.
  - String s = ”Learn Share Learn”;
  - int output = s.lastIndexOf("a"); // returns 14
  - String s = "Learn Share Learn"
  - int output = s.indexOf(“Play”); // return -1

Methods of String class

boolean equals(Object otherObj)

Compares this string to the specified object.
Boolean out = “Geeks”.equals(“Geeks”); // returns true
Boolean out = “Geeks”.equals(“geeks”); // returns false

boolean equalsIgnoreCase (String anotherString)

Compares string to another string, ignoring case considerations.
Boolean out= “Geeks”.equalsIgnoreCase(“Geeks”); // returns true
Boolean out = “Geeks”.equalsIgnoreCase(“geeks”); // returns true

String trim()

Returns the copy of the String, by removing whitespaces at both ends. It does not affect whitespaces in the middle.
String word1 = “ Learn Share Learn “;
String word2 = word1.trim(); // returns “Learn Share Learn”

String replace (char oldChar, char newChar)

Returns new string by replacing all occurrences of oldChar with newChar.
String s1 = “feeksforfeeks“;
String s2 = “feeksforfeeks”.replace(‘f’ ,’g’); // return “geeksforgeeks”

boolean contains(CharSequence sequence)

Returns true if string contains contains the given string.
sequence: This is an instance of CharSequence, which is an interface in Java representing a readable sequence of char values. Common implementations include String, StringBuilder, and StringBuffer.
String s1="geeksforgeeks";
String s2="geeks";
s1.contains(s2) // return true

StringBuffer

- StringBuffer is a class in Java that represents a mutable sequence of characters.
- StringBuffer objects are mutable, meaning that you can change the contents of the buffer without creating a new object.
- The initial capacity of a StringBuffer can be specified when it is created, or it can be set later with the ensureCapacity() method.
  - StringBuffer(int capacity): creates an empty string buffer with the specified capacity as length.

```
// Create a StringBuffer with an initial capacity of 20
StringBuffer sb = new StringBuffer(20);
```

- The append() method is used to add characters, strings, or other objects to the end of the buffer.
- The insert() method is used to insert characters, strings, or other objects at a specified position in the buffer.

StringBuffer example

```
public class StringBufferExample {
    public static void main(String[] args)
    {
        StringBuffer sb = new StringBuffer();
        sb.append("Hello");
        sb.append(" ");
        sb.append("world");
        String message = sb.toString();
        System.out.println(message);   // Hello world
    }
}
```

```
import java.io.*;

class A {
    public static void main(String args[])
    {
        StringBuffer sb = new StringBuffer("Hello ");
        sb.insert(1, "Java");
        // Now original string is changed
        System.out.println(sb);  // HJavaello
    }
}
```

Advantages of StringBuffer over String

- **There are several advantages of using StringBuffer over regular String objects in Java:**
- **Mutable**: StringBuffer objects are mutable, which means that you can modify the contents of the object after it has been created. In contrast, String objects are immutable, which means that you cannot change the contents of a String once it has been created.
- **Efficient**: Because StringBuffer objects are mutable, they are more efficient than creating new String objects each time you need to modify a string. This is especially true if you need to modify a string multiple times, as each modification to a String object creates a new object and discards the old one.

Methods of StringBuffer class

- **append() method**
- The append() method concatenates the given argument with this string.
- StringBuffer sb = new StringBuffer("Hello ");
- sb.append("Java"); // now original string is changed
- System.out.println(sb); //O/P: Hello Java
- **insert() method**
- The insert() method inserts the given string with this string at the given position.
- StringBuffer sb = new StringBuffer("Hello ");
- sb.insert(1, "Java");
- // Now original string is changed
- System.out.println(sb); // O/P: HJavaello

Methods of StringBuffer class

- **replace() method**
- The replace(i,j,String s) method replaces the given string from the specified beginIndex and endIndex-1.
- StringBuffer sb = new StringBuffer("Hello");
- sb.replace(1, 3, "Java");
- System.out.println(sb); // O/P: Hjavalo
- **delete() method**
- The delete(i,j) method of the StringBuffer class deletes the string from the specified beginIndex to endIndex-1.
- StringBuffer sb = new StringBuffer("Hello");
- sb.delete(1, 3);
- System.out.println(sb); // O/P: Hlo
- **reverse() method**
- The reverse() method of the StringBuilder class reverses the current string.
- StringBuffer sb = new StringBuffer("Hello");
- sb.reverse();
- System.out.println(sb); // O/P: olleH

Methods of StringBuffer class

- **Char charAt(int index):** Retrieves the character at the specified index
  - StringBuffer sb = new StringBuffer("Hello");
  - char ch = sb.charAt(1); // Gets the character at index 1.
- **Void setCharAt(int index, char ch)** : Sets the character at the specified index to the given character ch
  - StringBuffer sb = new StringBuffer("Hello");
  - sb.setCharAt(1, 'a'); // Changes character at index 1 to 'a‘ O/P: Hallo
- **length():** returns length of the StringBuffer object.
  - StringBuffer sb = new StringBuffer("Hello, World!");
  - int length = sb.length(); // returns 13
- **Void ensureCapacity(int minCapacity)** : ensure that the capacity of the StringBuffer is at least equal to the specified minimum capacity.
  - StringBuffer sb = new StringBuffer();
  - sb.ensureCapacity(50);

Methods of StringBuffer class

- **capacity() method**
  - The capacity() method of the StringBuffer class returns the current capacity of the buffer. The default capacity of the buffer is 16. If the number of characters increases from its current capacity, it increases the capacity by (oldcapacity*2)+2.
  - For instance, if your current capacity is 16, it will be (16*2)+2=34.

```
StringBuffer sb = new StringBuffer();
        System.out.println(sb.capacity()); // default 16
        sb.append("Hello");
        System.out.println(sb.capacity()); // now 16
        sb.append("java is my favourite language");
        System.out.println(sb.capacity());   // Now (16*2)+2=34     i.e
(oldcapacity*2)+2
```

Methods of StringBuffer class

- void getChars(int *sourceStart*, int *sourceEnd*, char *target*[ ], int *targetStart*) : Copy characters from the StringBuffer into a specified character array.
  - *sourceStart* : The starting index (inclusive) of the characters to be copied from the StringBuffer.
  - *sourceEnd* : The ending index (exclusive) of the characters to be copied.
  - target: The destination character array where the characters will be copied.
  - targetStart: The starting index in the destination array where copying will begin.

```
public class Main {
    public static void main(String[] args) {
        StringBuffer sb = new StringBuffer("Hello, World!");
        char[] dst = new char[5];  // Array to hold copied characters
        // Copy characters from StringBuffer to character array
        sb.getChars(0, 5, dst, 0);  // Copy "Hello" to dst starting at index 0
        // Convert the character array to a string for display
        String copiedString = new String(dst);
        System.out.println("Copied characters: " + copiedString);  // Output: Hello
    }
}
```

Enumerated type

- A Java enumeration is class type which serve the purpose of representing a group of named constants.

```
enum Level {
  LOW,
  MEDIUM,
  HIGH
}
```

- Accessing of enum constants : Level myVar = Level.MEDIUM;
- we don’t need to instantiate an enum using **new,** it has the same capabilities as other classes.
- Note: unlike classes, enumerations neither inherit other classes nor can get extended(i.e become superclass).

Declaration of enum

- **outside the class**

```
enum Color {
    RED,
    GREEN,
    BLUE;
}

public class Test {
public static void main(String[] args) {
        Color c1 = Color.RED;
        System.out.println(c1);
    }
}
```

Declaration of enum

- **inside a class**

```
public class Test {
    enum Color {
        RED,
        GREEN,
        BLUE;
    }

    public static void main(String[] args) {
        Color c1 = Color.RED;
        System.out.println(c1);
    }
}
```

Properties of enum

- **Class Type:** Every enum is internally implemented using the Class type.
- **Enum Constants:** Each enum constant represents an object of type enum.
- **Switch Statements:** Enum types can be used in switch statements.
- **Implicit Modifiers:** Every enum constant is implicitly public static final. Since it is static, it can be accessed using the enum name. Since it is final, enums cannot be extended.
- **Main Method:** Enums can declare a main() method, allowing direct invocation from the command line.

Enum in switch

```
import java.util.Scanner;
enum Day {
    SUNDAY,
    MONDAY,
    TUESDAY,
    WEDNESDAY,
    THURSDAY,
    FRIDAY,
    SATURDAY;
}
public class Test {
    Day day;
    // Constructor
    public Test(Day d) {
        day = d;
    }
public void dayIsLike() {
        switch (day) {
        case MONDAY:
            System.out.println("Mondays are bad.");
            break;
        case FRIDAY:
            System.out.println("Fridays are better.");
            break;
        case SATURDAY:
        case SUNDAY:
            System.out.println("Weekends are best.");
            break;
        default:
            System.out.println("Midweek days are so-so.");
            break;
        }
    }
public static void main(String[] args) {
        String str = "MONDAY";
        Test t1 = new Test(Day.valueOf(str));
        t1.dayIsLike();
    }
}
```

Main Function Inside Enum

- Enums can have a main function, allowing them to be invoked directly from the command line.

```
public enum Color {
    RED,
    GREEN,
    BLUE;
public static void main(String[] args) {
        Color c1 = Color.RED;
        System.out.println(c1);
    }
}
```

Loop through Enum

- We can iterate over the Enum using the values() method, which returns an array of enum constants.

```
import java.io.*;
enum Color {
   RED,
   GREEN,
   BLUE;
}
class GFG {
public static void main(String[] args) {
for (Color var_1 : Color.values()) {
          System.out.println(var_1);
      }
   }
}
```

Can an enum have a constructor and methods?

- Yes, an enum in Java can have constructors, methods, and even fields. The constructor is implicitly private.

```
enum Color {
    RED("Red"), GREEN("Green"), BLUE("Blue");
    private String colorName;
    private Color(String colorName) {  // Constructor
        this.colorName = colorName;
    }
public String getColorName() {  // Method
        return this.colorName;
    }
}
public class EnumExample {
    public static void main(String[] args) {
        Color color = Color.RED;
        System.out.println(color.getColorName());  // Output: Red
    }
}
```

Can we have a class with ‘private’ access modifier?

- In Java, classes cannot have the private access **modifier when they are declared at the top-level** (i.e., outside of another class). Top-level classes can only have two access modifiers: public & default.
- However, inner classes (classes defined within another class) can be declared as private. In this case, the inner class will only be accessible within the enclosing outer class.

```
// This will result in a compile-time error
private class MyClass {
    // Some code
}
```

```
public class OuterClass {
    private class InnerClass {
        // Some code
    }

    public void accessInner() {
        InnerClass obj = new InnerClass(); // This is valid
    }
}

// This is not allowed outside OuterClass:
// OuterClass.InnerClass obj = new OuterClass.InnerClass();
// Error
```

Questions

```
class Test {
    void display(int a, float b) {
        System.out.println("int-float");
    }
    void display(float a, int b) {
        System.out.println("float-int");
    }
public static void main(String args[]) {
        Test obj = new Test();
        obj.display(10, 10);
    }
}
```
OP: Ambiguous method call

```
class OverloadingTest {
    static void show(double d) {
        System.out.println("double");
    }
    static void show(String s) {
        System.out.println("String");
    }
    public static void main(String[] args) {
        show('A');
    }
}
```
OP: double

```
class OverloadTest {
    void test(Object obj) {
        System.out.println("Object");
    }
    void test(String str) {
        System.out.println("String");
    }
public static void main(String[] args) {
        OverloadTest obj = new OverloadTest();
        obj.test(null);
    }
}
```
OP: String

```
class OverloadingExample {
    void show(int... a) {
        System.out.println("Varargs");
    }
    void show(Integer a) {
        System.out.println("Integer");
    }
    public static void main(String[] args) {
        OverloadingExample obj = new
OverloadingExample();
        obj.show(10);
    }
}
```
OP: Integer

```
class Test {
    public static void main(String[] args) {
        int[] arr = new int[5];
        System.out.println(arr[3]);
    }
}
```
OP: 0

```
class Test {
    public static void main(String[] args
        int arr[] = {1, 2, 3, 4, 5};
        System.out.println(arr[arr[2]]);
    }
}
```
OP: 4

Questions

```
class Test {
    public static void main(String[] args) {
        int[] arr = new int[-5];
        System.out.println("Array
created!");
    }
}
```
OP: Runtime Error

```
class Test {
    public static void main(String[] args) {
        int arr1[] = {1, 2, 3};
        int arr2[] = {1, 2, 3};
        System.out.println(arr1 == arr2);
    }
}
```
OP: False

```
class Test {
    public static void main(String[] args) {
        int[][] arr = new int[2][];
        arr[0] = new int[3];
        arr[1] = new int[5];
        System.out.println(arr.length + " " +
arr[1].length);
    }
}
```
OP: 2 5

```
class Test {
    public static void main(String[] args) {
        int[] arr = {1, 2, 3, 4, 5};
        arr = null;
        System.out.println(arr[2]);
    }
}
```
OP: NullPointerException

```
class Test {
    public static void main(String[] args) {
        int[] arr = new int[3];
        arr[0] = 1;
        arr[1] = 2;
        arr = new int[5];
        System.out.println(arr[1]);
    }
}
```
OP: 0

```
class Test {
    public static void main(String[] args) {
        int[] arr = {10, 20, 30, 40, 50};
        int i = 0;
        arr[i++] = arr[i++] + arr[i++];
        System.out.println(arr[0]);
    }
}
```
OP: 50

Questions

```
String str1 = "Java";
String str2 = new String("Java");
String str3 = str2.intern();
System.out.println(str1 == str3);
```

OP: True

```
String str = "Java";
str += " Programming";
str.replace("Java", "Python");
System.out.println(str);
```

OP: Java Programming

```
String str = null;
System.out.println(str.valueOf(10));
```

OP: 10

```
String s = "abc";
System.out.println(s.substring(2, 1));
```

OP: Runtime exception

```
import java.util.*;public class Main {
 public static void main(String[] args) {
String s1 = "abc";
String s2 = "abc" + "def";
String s3 = s1 + "def";
System.out.println(s2 == s3);  }}
```

OP: False

```
public class Main {
   public static void main(String[] args) {
      String str1 = "Java";
      String str2 = "Java";
      String str3 = new String("Java");
      System.out.println(str1 == str2);
      System.out.println(str1 == str3);
      System.out.println(str1.equals(str3));
   }
}
```

OP:
true
false
true

Questions

```
class Test {
    int x;
    Test() {
        x = 10;
        System.out.println("Constructor: " + x);
    }
    {
        x = 20;
        System.out.println("Instance Block: " + x);
    }
    public static void main(String[] args) {
        Test obj = new Test();
    }
}
```

OP: Instance Block: 20
Constructor: 10

```
class Recursive {
    Recursive() {
        System.out.println("Calling itself...");
        new Recursive();
    }
    public static void main(String[] args) {
        Recursive obj = new Recursive();
    }
}
```

OP: Runtime error (StackOverflowError)

```
class Example {
    void display() {
        this();
        System.out.println("Hello");
    }

    public static void main(String[] args) {
        Example obj = new Example();
        obj.display();
    }
}
```

OP: Compile time error

```
class Sample {
    void show() {
        System.out.println(this);
    }
    public static void main(String[] args) {
        Sample obj = new Sample();
        obj.show();
    }
}
```

OP: Sample@78ac1102 (hashcode of obj)

```
class A {
    A getObject() {
        return this;
    }
    void display() {
        System.out.println("Hello from A");
    }
    public static void main(String[] args) {
        A obj = new A().getObject();
        obj.display();
    }
}
```

OP: Hello from A

Note: ‘this’ can be used to return an object in Java. It allows a method to return the current instance of the class.